

Benchmark: 2D Hatano-Nelson model

This example benchmarks `pyGBZ2d` using the two-dimensional Hatano-Nelson (HN) model. Closed-form expressions for its amoebic generalized Brillouin zone (GBZ) and strip GBZs provide a reference for checking the numerical results [1] (see arXiv:2506.22743v3).

The accompanying Python script demonstrates how to:

- Construct a characteristic polynomial from coefficients and exponents.
- Compute amoebic and strip-GBZ subsets at a reference energy E_{ref} .
- Read and visualize `PointSubset` and `LineSubset` data.
- Compare every returned sample with the closed-form GBZ and check its energy residual.
- Check whether an empty or nonempty result agrees with the analytic spectrum.
- Locate the spectrum with a coarse amoeba scan, then sample all four GBZs on a finer energy grid.

Run the script from the repository root with `python application/benchmark-2D-Hatano-Nelson.py`. It uses the local `src` directory by default and requires NumPy, SciPy, and Matplotlib for plotting. Add `--compare-only` to run the numerical checks without importing Matplotlib or opening figures. Set `RUN_IN_SRC = False` to use an installed copy of `pyGBZ2d` instead.

Introduction

The Hamiltonian of the 2D HN model is

$$H = \sum_{i,j} \left(J_{x1} c_{i+1,j}^\dagger c_{i,j} + J_{x2} c_{i-1,j}^\dagger c_{i,j} + J_{y1} c_{i,j+1}^\dagger c_{i,j} + J_{y2} c_{i,j-1}^\dagger c_{i,j} \right),$$

where $c_{i,j}$ annihilates a particle at site (i, j) and the hopping amplitudes are complex. Throughout this benchmark, all four hopping amplitudes are nonzero. Each pair can be written as

$$J_{\alpha 1} = e^{\gamma_\alpha + i\delta_\alpha} J_\alpha, \quad J_{\alpha 2} = e^{-\gamma_\alpha + i\delta_\alpha} J_\alpha^*, \quad \alpha = x, y$$

where $\gamma_\alpha, \delta_\alpha \in \mathbb{R}$ describe the hopping asymmetry and common phase, respectively. The pair J_α, J_α^* forms a Hermitian hopping pair before these factors are applied. A branch convention for this decomposition is

$$\begin{aligned} \gamma_\alpha &= \frac{1}{2} \ln \left(\frac{|J_{\alpha 1}|}{|J_{\alpha 2}|} \right) \\ \delta_\alpha &= \frac{1}{2} \text{Arg} (J_{\alpha 1} J_{\alpha 2}) \\ J_\alpha &= J_{\alpha 1} / e^{\gamma_\alpha + i\delta_\alpha} \end{aligned}$$

Here, `Arg` denotes the principal argument, as implemented by `numpy.angle`. The decomposition has a sign ambiguity: replacing $(\delta_\alpha, J_\alpha)$ by $(\delta_\alpha + \pi, -J_\alpha)$ leaves both hopping amplitudes unchanged. The convention above fixes a representative. The script implements it in `factorize_coefficients` and checks reconstruction in `check_factorization`.

The x -strip, y -strip, and amoebic GBZs coincide when expressed in the same Cartesian coordinates:

$$\beta_x = \exp(\gamma_x + i\theta_x),$$

$$\beta_y = \exp(\gamma_y + i\theta_y),$$

where $\theta_x, \theta_y \in [0, 2\pi)$. The $[11]$ -strip GBZ generally differs. In the basis $(\mathbf{a}_{[11]}, \mathbf{a}_y)$, with $\mathbf{a}_{[11]} = \mathbf{a}_x + \mathbf{a}_y$, it is

$$\tilde{\beta}_{[11]} = e^{\gamma_x + \gamma_y + i\theta_{[11]}},$$

$$\tilde{\beta}_y = e^{\gamma_y + i\theta_y} \sqrt{\frac{J_x^* e^{i\Delta_{xy} + i\theta_{[11]}} + J_y}{J_x e^{i\Delta_{xy} - i\theta_{[11]}} + J_y^*}},$$

where $\Delta_{xy} := \delta_x - \delta_y$. These expressions follow Eqs. (S3.16) and (S3.30) of Ref. [1]. The second expression applies where its numerator and denominator are nonzero. When $\sin(\Delta_{xy}) \neq 0$, the transverse radius depends on $\theta_{[11]}$, and the model exhibits geometry-dependent bands. When $\sin(\Delta_{xy}) = 0$, the radius reduces to $\exp(\gamma_y)$ at nonsingular phases.

The corresponding energy spectra are illustrated in Figure 1.

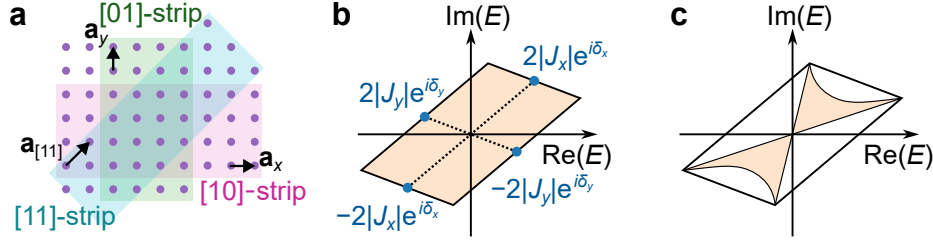


Figure 1: Strip geometries and GBZ spectra of the 2D HN model. **a** The x -, y -, and $[11]$ -strip geometries. **b** The common spectrum of the x -strip, y -strip, and amoebic GBZs. **c** The $[11]$ -strip GBZ spectrum.

Modeling with characteristic polynomials

`pyGBZ2d` represents a tight-binding model through its characteristic Laurent polynomial $\det(EI - h(\beta_1, \beta_2))$. Write its expansion as

$$f(E, \beta_1, \beta_2) = \sum_{j=1}^{N_{\text{terms}}} a_j E^{l_j} \beta_1^{m_j} \beta_2^{n_j}, \quad l_j \in \mathbb{N}, m_j, n_j \in \mathbb{Z}$$

where l_j is a nonnegative integer. The coefficient array has shape `(N_terms, )`, and the integer degree array has shape `(N_terms, 3)`. Their entries correspond term by term. Schematically,

```

from pygbz2d import core
import numpy as np

coeffs = np.array([a1, a2, ..., aN], dtype=complex)
degs = np.array([
    [l1, m1, n1],
    [l2, m2, n2],
    ...,
    [lN, mN, nN],
], dtype=int)
poly = core.CharPoly(coeffs, degs)

```

Changing the lattice basis changes the coordinates used to describe a GBZ. Under an invertible integer change of lattice basis, the amoebic GBZ describes the same physical set after transforming the coordinates back. Strip GBZs additionally depend on the choice of major axis, so the two input Bloch factors play different roles.

The `pygbz2d.sgbz` module assumes that the polynomial is **expressed in the basis $(\mathbf{a}_1, \mathbf{a}_2)$** , with $\mathbf{a}_1$ as the **major axis**. A change of major axis can therefore change the physical GBZ, even after both results are expressed in Cartesian coordinates.

For the Cartesian basis $(\mathbf{a}_x, \mathbf{a}_y)$, the Bloch Hamiltonian is

$$h(\beta_x, \beta_y) = J_{x1}\beta_x^{-1} + J_{x2}\beta_x + J_{y1}\beta_y^{-1} + J_{y2}\beta_y,$$

and the characteristic polynomial is

$$f(E, \beta_x, \beta_y) = E - J_{x1}\beta_x^{-1} - J_{x2}\beta_x - J_{y1}\beta_y^{-1} - J_{y2}\beta_y.$$

For a y -strip calculation, interchange the last two columns of the degree array so that $(\beta_1, \beta_2) = (\beta_y, \beta_x)$.

For $(\mathbf{a}_{[11]}, \mathbf{a}_y)$, the coordinate transformation is

$$\beta_{[11]} = \beta_x \beta_y, \quad \beta_x = \frac{\beta_{[11]}}{\beta_y}.$$

Substitution gives

$$f(E, \beta_{[11]}, \beta_y) = E - J_{x1}\beta_{[11]}^{-1}\beta_y - J_{x2}\beta_{[11]}\beta_y^{-1} - J_{y1}\beta_y^{-1} - J_{y2}\beta_y.$$

The function `get_HN_charpoly` implements all three choices using `which="x-y"`, `"y-x"`, or `"11-y"`. It returns `(coeffs, degs)`, which can be passed directly to either solver; constructing a `CharPoly` explicitly is optional for these entry points.

For larger models, entering coefficient and degree arrays manually can become cumbersome. The `TightBinding` module of `BerryPy` offers an alternative interface based on hopping terms. The present benchmark uses explicit arrays to make the coordinate conventions transparent and requires no additional modeling package.

Computing GBZ subsets at a reference energy

Both `sgbz` and `amoeba` provide `collect_GBZ_subsets` to compute the intersection of the GBZ with $f(E_{\text{ref}}, \beta_1, \beta_2) = 0$. This is the non-Hermitian counterpart of an equal-energy contour, although the

intersection may consist of isolated points. The functions share the basic inputs `coeffs`, `degs`, and `E_ref`, and both return a `GBZResult`:

```
from pygbz2d import amoeba, sgbz

coeffs, degs = get_HN_charpoly(
    1 + 1j, 1.5 + 1.2j, -1 + 1j, -1.2 - 0.5j, "x-y",
)
res_amoeba = amoeba.collect_GBZ_subsets(coeffs, degs, 1 + 1j)
res_xstrip = sgbz.collect_GBZ_subsets(coeffs, degs, 1 + 1j)
```

Their optional parameters differ. For example, `sgbz` accepts `mu1_guess`, whereas `amoeba` accepts `mu1_low`, `mu1_high`, `mu2_low`, and `mu2_high`. Both accept `debug_mode=True`, which propagates solver exceptions for diagnosis. Progress reporting belongs to the scan helper; the library entry points do not accept `perc`. All numerical options are explicitly named; there is no catch-all keyword dictionary. See `doc/constants.md` for the live default settings.

■ Reading the result

`GBZResult`, defined in `src/pygbz2d/core.py`, stores the reference energy, solver status, and connected subsets:

Attribute	Meaning
<code>E_ref</code>	Reference energy supplied to the solver.
<code>success</code> , <code>error</code>	Whether the computation completed without error, and an error message if it did not.
<code>subsets</code>	A list of <code>PointSubset</code> and/or <code>LineSubset</code> objects.
<code>index</code>	The pair (n_{0D}, n_{1D}) counting point and line subsets.
<code>is_empty</code>	Whether <code>subsets</code> is empty, regardless of solver status.
<code>is_gbz</code>	Whether <code>success</code> is true and <code>index</code> differs from $(0, 0)$.

Always check `success` before interpreting an empty result. A failed calculation does not establish that the energy lies outside the spectrum:

```
res = res_xstrip
if not res.success:
    raise RuntimeError(res.error)
if res.is_empty:
    print("No GBZ subsets found at this reference energy.")
else:
    print(res.index) # (number of point subsets, number of line subsets)
```

The subset data fields are summarized below; validation methods and convenience properties are omitted:

```

@dataclass(frozen=True)
class PointSubset:
    """An isolated point in the solver's coordinate basis."""
    E: complex
    beta1: complex
    beta2: complex

@dataclass
class LineSubset:
    """One sampled curve with constant  $\mu_1 = \ln|\beta_{\text{e1}}|$ ."""
    E: complex
    mu1: float
    theta1_arr: np.ndarray # shape (N,), ordered phase samples
    beta2_arr: np.ndarray # shape (N,), eagerly stored Bloch factors

```

For a line subset, reconstruct the first Bloch factor using

```

beta1_arr = np.exp(subset.mu1 + 1j * subset.theta1_arr)
beta2_arr = subset.beta2_arr

```

The number of stored samples is distinct from the number of line subsets. Each `LineSubset` represents one β_2 curve; a degeneracy can give multiple curve objects. `subset_samples` provides a common array interface for points and lines, and `calculate_subset_and_show` demonstrates solving, comparison, data retrieval, and visualization. Phases are plotted modulo 2π using dots to avoid artificial connections across the angular seam. The dashed radius curve describes the full analytic GBZ constraint, rather than only the points at E_{ref} .

■ Point and line examples

For the Cartesian GBZs, substituting the analytic radii into the Hamiltonian gives

$$E(\theta_x, \theta_y) = 2e^{i\delta_x} \operatorname{Re}(J_x e^{-i\theta_x}) + 2e^{i\delta_y} \operatorname{Re}(J_y e^{-i\theta_y}).$$

Their spectrum has nonzero area when $\sin(\Delta_{xy}) \neq 0$. At a regular energy in a two-dimensional spectral region, fixing a complex energy generally imposes two real constraints and leaves isolated GBZ points. When the spectrum collapses to a line, regular energies instead generally have continuous preimages. Boundary energies and degeneracies require separate consideration.

This area statement must be qualified for the [11] strip: when $|J_x| = |J_y|$, its spectrum can have zero area even if $\sin(\Delta_{xy}) \neq 0$, while the GBZ remains geometry dependent. See Sec. S6 of Ref. [1].

The script uses two fixed parameter sets, each evaluated for all four GBZ choices:

- `demo_point_subsets`: $J_{x1} = 1 + i$, $J_{x2} = 1.5 + 1.2i$, $J_{y1} = -1 + i$, $J_{y2} = -1.2 - 0.5i$, and $E_{\text{ref}} = 1 + i$. The hopping parameters match the example in Ref. [1].
- `demo_line_subsets`: $J_{x1} = 1$, $J_{x2} = 1.5$, $J_{y1} = -1$, $J_{y2} = -1.2$, and $E_{\text{ref}} = 1$. Here $\delta_x = \delta_y = 0$ in the chosen convention, and the spectra are real.

■ Comparison to closed-form solutions

The comparison first checks whether the reference energy belongs to the analytic GBZ spectrum, including when the numerical result is empty. For a nonempty result, it then evaluates the analytic radius constraints

at every returned phase and independently checks $E_{\text{ref}} = h(\beta_x, \beta_y)$. Both sample checks are needed: satisfying the characteristic equation alone does not establish GBZ membership, and satisfying the radius constraints alone does not fix the energy.

■ Checking spectral membership, including empty results

Define the complex spectral half-widths

$$u = 2 |J_x| e^{i\delta_x}, \quad v = 2 |J_y| e^{i\delta_y}.$$

For the amoebic, x -strip, and y -strip GBZs, the dispersion relation gives

$$\sigma_{\text{Cartesian}} = \{us + vt : s, t \in [-1, 1]\}.$$

Thus the spectrum is the closed parallelogram with vertices $u + v$, $-u + v$, $-u - v$, and $u - v$. When u and v are collinear, this set reduces to a line segment. A point is outside if it lies beyond the segment endpoints or off the segment's supporting line.

For the [11] strip, write $h = A\beta_2 + \frac{B}{\beta_2}$, with $A = \frac{J_{x1}}{\beta_1} + J_{y2}$ and $B = J_{x2}\beta_1 + J_{y1}$. On its analytic GBZ, the equal-modulus condition gives $E = 2\sqrt{AB}s$ for $s \in [-1, 1]$. As the phase of β_1 varies, $4AB$ traverses the line segment joining $(u + v)^2$ and $(u - v)^2$. Squaring the energy therefore gives the following equivalent membership test:

$$E \in \sigma_{[11]} \iff E^2 \in \text{conv}\{0, (u + v)^2, (u - v)^2\},$$

where conv denotes the convex hull. This is a triangle in the squared-energy plane, possibly collapsed to a segment. Both signs of E occur, so squaring introduces no extra spectral branch. The test follows directly from the strip dispersion in Eq. (S3.31) of Ref. [1].

`closed_form_spectrum_membership` evaluates these sets independently of the numerical solver. It computes the distance to the closed polygon using edge projections and an interior test, including collapsed polygons; no angular sampling mesh is needed. Let $L = |u| + |v|$. The distance is normalized by L in the Cartesian energy plane and by L^2 in the [11] squared-energy plane. A normalized distance no larger than `spectrum_atol = 1e-8` is classified as inside. Points within this tolerance of a boundary are consequently treated as spectral points; the squared-energy tolerance has a different interpretation near $E = 0$ from a tolerance applied directly to E .

`compare_to_closed_form` compares this analytic classification with the numerical result:

Analytic classification	Numerical result	Comparison
Inside	Nonempty	Proceed to the radius and energy checks.
Outside	Empty	Pass the spectral-membership check; there are no samples to check.
Inside	Empty	Raise a spectrum-membership mismatch.
Outside	Nonempty	Raise a spectrum-membership mismatch.

A failed solve is always an error, regardless of the analytic classification. For a verified empty result, `n_samples` is zero and the three sample-error fields are `None` (printed as `N/A`), rather than zero. The report includes `analytic_is_gbz`, `numerical_is_gbz`, `spectrum_distance`, and `spectrum_energy_power`; the last field is 1 for a Cartesian spectrum and 2 for the [11] spectrum.

■ Coordinate conventions and analytic radii

Let $\mu_j = \ln|\beta_j|$ and $\theta_1 = \text{Arg}(\beta_1)$. `closed_form_log_radii` returns the following analytic values in the solver's own basis:

GBZ choice	(β_1, β_2)	μ_1^{exact}	μ_2^{exact}
amoeba / x-strip	(β_x, β_y)	γ_x	γ_y
y-strip	(β_y, β_x)	γ_y	γ_x
11-strip	$(\beta_x\beta_y, \beta_y)$	$\gamma_x + \gamma_y$	$\gamma_y + (\frac{1}{2}) \ln R(\theta_1) $

Here,

$$R(\theta_1) = \frac{J_x^* e^{i(\Delta_{xy} + \theta_1)} + J_y}{J_x e^{i(\Delta_{xy} - \theta_1)} + J_y^*}.$$

The [11] formula can also be checked directly: in this basis the Hamiltonian is $h = A\beta_2 + \frac{B}{\beta_2}$, where

$$A = \frac{J_{x1}}{\beta_1} + J_{y2}, \quad B = J_{x2}\beta_1 + J_{y1}.$$

The two roots of $A\beta_2^2 - E\beta_2 + B = 0$ have product $\frac{B}{A}$, so their equal-modulus condition gives $|\beta_2| = \sqrt{|\frac{B}{A}|}$. At $|\beta_1| = \exp(\gamma_x + \gamma_y)$, this reproduces the radius above. The benchmark requires finite, nonzero radii and reports singular expressions rather than clipping or replacing them. The two demonstration parameter sets have $|J_x| \neq |J_y|$, so neither the numerator nor the denominator of R vanishes for real θ_1 .

■ Error measures

For the N returned samples, define

$$\varepsilon_{\mu_j} = \max_{1 \leq n \leq N} |\ln|\beta_{j,n}| - \mu_j^{\text{exact}}(\theta_{1,n})|, \quad j = 1, 2.$$

For a `PointSubset`, there is one sample. For a `LineSubset`, all stored samples are used, with $\beta_{1,n} = \exp(\mu_1 + i\theta_{1,n})$.

To check the energy, first convert the samples to Cartesian coordinates: interchange the two components for a y strip, and use $\beta_x = \frac{\beta_1}{\beta_2}$, $\beta_y = \beta_2$ for a [11] strip. Define the four hopping contributions as

$$(t_{1,n}, t_{2,n}, t_{3,n}, t_{4,n}) = \left(\frac{J_{x1}}{\beta_{x,n}}, J_{x2}\beta_{x,n}, \frac{J_{y1}}{\beta_{y,n}}, J_{y2}\beta_{y,n} \right).$$

The dimensionless energy residual is

$$\varepsilon_E = \max_{1 \leq n \leq N} \frac{|E_{\text{ref}} - \sum_{q=1}^4 t_{q,n}|}{|E_{\text{ref}}| + \sum_{q=1}^4 |t_{q,n}|}.$$

This normalization uses the magnitudes of the individual hopping terms, so cancellation in their sum does not create an artificially small denominator. It also remains well defined at $E_{\text{ref}} = 0$ for the nonzero finite hoppings and Bloch factors considered here.

For a nonempty result with matching spectral membership, `compare_to_closed_form` returns the subset counts, sample count, and these three maximum errors. It rejects invalid samples and accepts the sample comparison when $\varepsilon_{\mu_1}, \varepsilon_{\mu_2} \leq 10^{-5}$ and $\varepsilon_E \leq 10^{-7}$. These are benchmark acceptance thresholds, separate from both `spectrum_atol` and the solver settings; they do not modify solver tolerances. Exceeding either

threshold raises an exception with the measured errors. The point and line demos also require nonempty results of their expected subset type.

▪ Running the comparison

From the repository root, run

```
python application/benchmark-2D-Hatano-Nelson.py --compare-only
```

The default `--mode demo` executes eight point/line comparisons and sixteen additional spectral-membership comparisons. The separate `check_factorization()` helper can be called explicitly to check hopping reconstruction. To inspect one result from within the accompanying script or an interactive session with its functions loaded, use

```
hoppings = (1 + 1j, 1.5 + 1.2j, -1 + 1j, -1.2 - 0.5j)
res = calculate_subsets(1 + 1j, *hoppings, "11-strip")
report = compare_to_closed_form(res, *hoppings, "11-strip")
print_comparison(report)
```

`demo_spectrum_membership` covers the following additional reference energies for all four GBZ choices. “Complex” and “real” refer to the hopping parameters in `demo_point_subsets` and `demo_line_subsets`, respectively:

Hoppings	E_{ref}	amoeba / x / y	11-strip
Complex	$6 + 6i$	Outside	Outside
Real	6	Outside: beyond endpoints	Outside: beyond endpoints
Real	i	Outside: off the real axis	Outside: off the real axis
Complex	$2 + 2i$	Inside	Outside

In the recorded benchmark run, all sixteen additional comparisons passed: thirteen returned verified empty results and three returned nonempty point subsets. The last case checks geometry-dependent membership explicitly. For example, replacing $1 + 1j$ by $2 + 2j$ in the code above gives a verified empty [11] result, with `index=(0, 0)`, `n_samples=0`, and `analytic_is_gbz=False`.

The optional `--mode benchmark --compare-only` mode uses random hoppings and samples an energy from the Cartesian GBZ. That guarantees Cartesian spectral membership, but does not guarantee [11]-strip membership. Its results are therefore judged by the same analytic membership test, rather than by requiring every solver result to be nonempty.

The following results were obtained on 2026-09-21 with that checkout, the NumPy polynomial backend (NumPy 1.26.4, SciPy 1.11.4), and the default solver settings. N counts all stored samples, including any shared endpoints. Adaptive sampling and future solver changes can alter N and the last digits of the errors.

Case / GBZ	index	N	$\varepsilon_{\mu 1}$	$\varepsilon_{\mu 2}$	ε_E
Point / amoeba	(4, 0)	4	2.00×10^{-9}	9.52×10^{-9}	4.79×10^{-16}
Point / x-strip	(4, 0)	4	2.00×10^{-9}	8.00×10^{-14}	1.00×10^{-16}
Point / y-strip	(4, 0)	4	1.44×10^{-9}	8.72×10^{-13}	1.00×10^{-16}
Point / 11-strip	(4, 0)	4	5.55×10^{-10}	1.44×10^{-9}	2.10×10^{-16}
Line / amoeba	(0, 2)	358	3.48×10^{-10}	8.45×10^{-7}	1.32×10^{-10}
Line / x-strip	(0, 2)	350	3.48×10^{-10}	4.22×10^{-7}	1.32×10^{-10}
Line / y-strip	(0, 2)	346	4.41×10^{-10}	5.80×10^{-7}	1.28×10^{-10}
Line / 11-strip	(0, 2)	380	1.42×10^{-10}	3.98×10^{-7}	5.51×10^{-11}

Table 1: Comparison with the analytic GBZ radii and the fixed-energy equation for the two demonstration parameter sets. All eight cases pass the stated thresholds.

Agreement means that numerical and analytic spectral membership match within `spectrum_atol` and, for nonempty results, every sampled point satisfies the closed-form GBZ constraints and fixed-energy equation within the sample tolerances. The independent membership test also checks empty results. It does not prove that every analytic branch was found in a nonempty result or that interpolation between line samples is equally accurate. The next section extends the calculation to a grid of reference energies.

Computation of full GBZ

The solvers return GBZ subsets at a specified reference energy. Scanning an energy region and collecting these subsets therefore produces a sampled representation of the GBZ. The spectrum of a strip GBZ is contained in the amoebic spectrum,

$$\sigma_{\text{strip}} \subseteq \sigma_{\text{amoeba}},$$

so a common scan region can be selected from an amoeba-only coarse scan. We use the complex hopping parameters from `demo_point_subsets` and perform two stages:

Stage	Energy grid	Methods	Output
Coarse	20×20	amoeba	Display the spectrum and retain results only in memory.
Fine	100×100	All four GBZs	Save complete results in <code>application/data</code> .

The Python section beginning with `Full GBZ sweep` lists its own imports, model parameters, and basis choices. Together with `get_HN_charpoly`, it can be copied into a separate script using an installed `pyGBZ2d`. The scan itself does not depend on the earlier plotting or closed-form comparison helpers. `sweep_GBZ` constructs the characteristic polynomial once for the selected basis; `_sweep_worker` directly calls `amoeba.collect_GBZ_subsets` or `sgbz.collect_GBZ_subsets` for each energy.

■ Coarse scan: determine the energy window

`coarse_sweep` uses 20 equally spaced points on each axis, including both endpoints of $[-5, 5]$. There are 400 reference energies, with spacing

$$\Delta E_{\text{Re}} = \Delta E_{\text{Im}} = \frac{10}{19}.$$

Only `amoeba.collect_GBZ_subsets` is used. The scan collects numerical results and displays the spectrum. Coarse results and the figure are not written to disk. The returned dictionary retains the grid and

results in memory so they can be passed directly to `fine_sweep`. Closed-form comparison is a separate step performed after scanning.

Let S be the coarse energies for which `result.is_gbz` is true. The fine-scan limits are

$$\begin{aligned} [E_{\text{Re, min}}, E_{\text{Re, max}}] &= \left[\min_{E \in S} \text{Re}(E) - \Delta E_{\text{Re}}, \max_{E \in S} \text{Re}(E) + \Delta E_{\text{Re}} \right], \\ [E_{\text{Im, min}}, E_{\text{Im, max}}] &= \left[\min_{E \in S} \text{Im}(E) - \Delta E_{\text{Im}}, \max_{E \in S} \text{Im}(E) + \Delta E_{\text{Im}} \right]. \end{aligned}$$

Thus each side of the occupied bounding box is extended by one **coarse-grid** spacing. The coarse plot shows this proposed rectangle as a dashed outline. Bounds come entirely from the numerical coarse results.

To inspect only the coarse scan, run from the repository root:

```
python application/benchmark-2D-Hatano-Nelson.py --mode coarse-sweep --n-process 20
```

The command above requests 20 processes; `N_PROCESS` sets the script's default process count. If no spectral points are detected, the scan cannot determine a fine window. If occupied points touch the coarse-grid boundary, the window may be truncated. Either condition is reported as an error rather than silently producing a fine scan. Even when neither occurs, a finite coarse grid is an estimate of the support; it cannot establish that arbitrarily narrow spectral features were resolved. In particular, the 20-point grid does not include zero, so it is not suited to the real-line spectrum of `demo_line_subsets` without changing the grid.

■ Fine scan: compute and save all four GBZs

`fine_sweep` defaults to 100 equally spaced points along each expanded interval. All four methods evaluate the same $100^2 = 10,000$ energies, for 40,000 fine-grid solves in total. Direct callers can change `grid_size`. No grid point is skipped based on an analytic classification or on another method's result. The function computes, saves, and optionally plots the numerical results; it does not require a closed-form solution.

The complete two-stage workflow is

```
python application/benchmark-2D-Hatano-Nelson.py --mode full-sweep --n-process 20
```

The coarse plot is displayed first. Closing that figure allows the fine scan to start. After completion, four panels display the fine-grid spectra. Starting this command after a separate `--mode coarse-sweep` run recomputes the coarse scan, since its data were not saved. For an unattended calculation with the same grids and output files, use

```
python application/benchmark-2D-Hatano-Nelson.py --mode full-sweep --n-process 20 --no-show
```

To call the functions directly from a script that has imported them, use a main guard so Windows can start the worker processes safely:

```
if __name__ == "__main__":
    coarse = coarse_sweep(n_process=20)
    fine = fine_sweep(coarse, n_process=20)
```

`sweep_GBZ` distributes independent energies across the requested worker processes, reports progress in the parent process, and restores the original energy order when workers finish out of order. Each spawned worker uses one BLAS thread. The hopping argument order is consistently `(Jx1, Jx2, Jy1, Jy2)`. An

exception propagated out of the library stops the scan and reports the GBZ choice, energy index, and reference energy. Returned failed `GBZResult` objects are retained and plotted as red crosses rather than exterior points; a coarse scan containing failures cannot be used to infer a fine window.

The default output directory is resolved relative to the Python script and is created with `makedirs(parents=True, exist_ok=True)` when needed. Each completed method is saved as

```
application/data/HN2D-<UTC run ID>-amoeba.pkl
application/data/HN2D-<UTC run ID>-x-strip.pkl
application/data/HN2D-<UTC run ID>-y-strip.pkl
application/data/HN2D-<UTC run ID>-11-strip.pkl
```

The shared run ID in the filenames groups the four files and keeps repeated runs from overwriting earlier results. The in-memory scan and every saved file use the same six-field dictionary:

Entries	Meaning
stage	"coarse" or "fine", used for the plot title and the coarse refinement rectangle.
hoppings	A dictionary with keys <code>Jx1</code> , <code>Jx2</code> , <code>Jy1</code> , and <code>Jy2</code> . Parameter names carry their meaning without a separate ordering field.
real_axis	The one-dimensional $\text{Re}(E)$ grid.
imag_axis	The one-dimensional $\text{Im}(E)$ grid.
flatten_order	Flattening order, "C" for newly computed scans. Grid rows correspond to $\text{Im}(E)$ and columns to $\text{Re}(E)$.
results	A mapping from GBZ names to lists of complete <code>GBZResult</code> objects, including empty or failed results and all stored line samples.

The returned fine scan has four entries in `results`; each per-method file has one entry, such as `{"11-strip": [...]}`. Both can be passed directly to `plot_sweep_spectra`. Energy arrays, grid shapes, and coarse refinement bounds are derived when needed, rather than stored alongside the axes. Coarse setup metadata, process counts, saved paths, and filename identifiers are not part of the scan data.

Files are written after each method completes, and their paths are printed. If a later method raises an exception, previously completed files remain available; an unfinished method is not published as a completed scan.

■ Validate the completed scan against the HN solution

The separate `compare_sweep_to_closed_form` helper belongs to the earlier benchmark code. It applies `compare_to_closed_form` to the stored results after scanning, including empty results, without rerunning the solvers:

```
# Optional HN-specific validation after the generic scanning workflow.
coarse_reports = compare_sweep_to_closed_form(coarse)
fine_reports = compare_sweep_to_closed_form(fine)
```

The command-line demo performs this validation automatically after the requested scans finish: after the coarse scan for `--mode coarse-sweep`, or after both scans and all four fine-scan files have been saved for `--mode full-sweep`. With figures enabled, close the scan figures to proceed to validation. A mismatch reports the method, energy index, and energy while preserving the computed results and saved files. A completed numerical file does not itself imply that analytic validation passed. For a model without a closed-form solution, the scanning workflow can be used on its own and this benchmark-specific step omitted.

■ Reading the saved scan

`load_sweep` reads the saved dictionary without conversion. Its result can be plotted immediately, just like the object returned by `fine_sweep`. Replace the filename below with the path printed by the scan:

```
import numpy as np
from pygbz2d import PointSubset, LineSubset

data = load_sweep("application/data/HN2D-<UTC run ID>-1l-strip.pkl")
plot_sweep_spectra(data)

results = data["results"]["1l-strip"]
energy_grid = (
    data["real_axis"][None, :] + 1j * data["imag_axis"][:, None]
)
energies = energy_grid.ravel(order=data["flatten_order"])
mask = np.array([r.is_gbz for r in results])
spectral_energies = energies[mask]
membership_grid = mask.reshape(
    energy_grid.shape, order=data["flatten_order"],
)

for result in results:
    for subset in result.subsets:
        if isinstance(subset, PointSubset):
            beta1, beta2 = subset.beta1, subset.beta2
        elif isinstance(subset, LineSubset):
            beta1 = np.exp(subset.mu1 + 1j * subset.theta1_arr)
            beta2 = subset.beta2_arr
        # [1l]-strip coordinates -> Cartesian coordinates.
        beta_x, beta_y = beta1 / beta2, beta2
```

Alternatively, the command-line plot mode reads a saved scan without rerunning either solver:

```
python application/benchmark-2D-Hatano-Nelson.py --mode plot --data-fname <saved-
file.pkl>
```

`GBZResult` and its subset classes remain defined in `pygbz2d.core`, so that package must be importable when loading the files. The energy grid samples the spectral region, while the stored subset objects provide the corresponding GBZ coordinates. A finite scan is a sampled approximation of the full GBZ, with resolution set by both the energy grid and the solver's adaptive sampling of any line subsets.

Bibliography

- [1] C. Wang, J. Pi, Q. Liu, Y. Li, and Y.-C. Liu, General Theory for Geometry-Dependent Non-Hermitian Bands, Arxiv (2025).